

Sistema de Gestión de Expedientes

SGE — Trabajo Práctico N.º 1

Seminario de Lenguajes opción .NET — 1.º Semestre 2026

Documento Explicativo

Integrantes:

Cazalá Franco	Legajo: 02700/7
Maranzana Valentina	Legajo: 01917/9
Medina Alan	Legajo: 01551/9
Laso Felipe	Legajo: 02703/1

Mayo 2026

Índice

1	Compilación y ejecución	2
2	Composition Root	2
3	Camino Feliz	3
3.1	Prueba 1 — Agregar un expediente válido	3
3.2	Prueba 2 — Listar todos los expedientes	3
3.3	Prueba 3 — Agregar un trámite y verificar cambio de estado automático .	4
3.4	Prueba 4 — Cambiar estado manualmente	5
3.5	Prueba 5 — Modificar carátula	6
4	Camino de Error	7
4.1	Prueba 6 — Carátula vacía lanza <code>DominioException</code>	7
4.2	Prueba 7 — ID inexistente lanza <code>EntidadNoEncontradaException</code>	7
4.3	Prueba 8 — <code>AutorizacionException</code> al denegar permisos	8

Compilación y ejecución

Para compilar y ejecutar el proyecto, abrir una terminal en la raíz de la solución (donde se encuentra el archivo `SGE.slnx`) y correr los siguientes comandos:

```
# Compilar toda la solución
dotnet build

# Ejecutar el proyecto de consola
dotnet run --project SGE.Consola
```

Código 1: Compilación y ejecución

Al ejecutarse, el programa crea automáticamente los archivos `expedientes.txt` y `tramites.txt` en el directorio `SGE.Consola/` si no existen. Cada vez que se corre, los datos se acumulan en esos archivos. Para empezar desde cero, eliminarlos manualmente antes de ejecutar.

Composition Root

Lo primero que hace `Program.cs` es instanciar todas las dependencias concretas de Infraestructura e implementadas en los Casos de Uso. Este es el único lugar de todo el sistema donde se crean instancias concretas; el resto del código solo conoce interfaces.

```
// Infraestructura
var expedienteRepository = new ExpedienteTxtRepository();
var tramiteRepository    = new TramiteTxtRepository();
var autorizacionService  = new AutorizacionProvisionalService();
var actualizarEstado     = new ActualizacionEstadoExpedienteService(
    expedienteRepository, tramiteRepository);

// Casos de uso de Expedientes
var agregarExpediente = new AgregarExpedienteUseCase(
    expedienteRepository, autorizacionService);
var eliminarExpediente = new BajaExpedienteUseCase(expedienteRepository,
    tramiteRepository, autorizacionService);
var modificarCaratula = new ModificarCaratulaExpedienteUseCase(
    expedienteRepository, autorizacionService);
var cambiarEstado     = new CambiarEstadoExpedienteUseCase(
    expedienteRepository, autorizacionService);
var listarExpedientes = new ListarExpedientesUseCase(
    expedienteRepository);

// Casos de uso de Tramites
var agregarTramite = new AgregarTramiteUseCase(tramiteRepository,
    expedienteRepository, autorizacionService, actualizarEstado);
var eliminarTramite = new BajaTramiteUseCase(tramiteRepository,
    autorizacionService, actualizarEstado);
var modificarTramite = new ModificarTramiteUseCase(tramiteRepository,
    autorizacionService, actualizarEstado);
var listarTramites = new ListarTramitesPorExpedienteUseCase(
    tramiteRepository);
```

```
var idUsuario = Guid.NewGuid(); // Simulamos un usuario logueado
```

Código 2: Creación de dependencias

Camino Feliz

Prueba 1 — Agregar un expediente válido

Se arma un `AgregarExpedienteRequest` con una carátula válida y el ID del usuario simulado, y se lo pasa a `agregarExpediente.Ejecutar()`. Si todo sale bien, se imprime el ID del expediente recién creado.

```
Console.WriteLine("== 1. Agregar un expediente valido ==");

try
{
    var request = new AgregarExpedienteRequest(
        "Expediente de prueba",
        idUsuario);

    var response = agregarExpediente.Ejecutar(request);

    Console.WriteLine(
        $"[Exito] Expediente creado. Id: {response.IdExpediente}\n");
}
catch (DominioException ex)
{
    Console.WriteLine($"[DominioException]: {ex.Message}\n");
}
catch (AutorizacionException ex)
{
    Console.WriteLine($"[AutorizacionException]: {ex.Message}\n");
}
catch (Exception ex)
{
    Console.WriteLine($"[Exception]: {ex.Message}\n");
}
```

Código 3: Alta de expediente

```
== 1. Agregar un expediente valido ==
[Exito] Expediente creado. Id: 3f2504e0-4f89-11d3-9a0c-0305e82c3301
```

Imagen 1: Salida esperada

```
== 1. Agregar un expediente valido ==
[AutorizacionException]: El usuario no tiene permisos para crear
expedientes.
```

Imagen 2: Escenario sin permisos

Prueba 2 — Listar todos los expedientes

Se llama a `listarExpedientes.Ejecutar()` que devuelve una colección de DTOs. Se recorre con `foreach` e imprime el ID, la carátula y el estado de cada uno.

```
Console.WriteLine("== 2. Listar expedientes ==");

try
{
    var expedientes = listarExpedientes.Ejecutar();

    foreach (var e in expedientes)
        Console.WriteLine(
            $"> Id: {e.Id} | Caratula: {e.Caratula} | Estado: {e.Estado}"
        );

    Console.WriteLine();
}
catch (Exception ex)
{
    Console.WriteLine($"[Exception]: {ex.Message}\n");
}
```

Código 4: Listado de expedientes

```
== 2. Listar expedientes ==
> Id: 3f2504e0-4f89-11d3-9a0c-0305e82c3301 | Caratula: Expediente de
prueba | Estado: RecienIniciado
```

Imagen 3: Salida esperada

Prueba 3 — Agregar un trámite y verificar cambio de estado automático

Se agrega un trámite con etiqueta PaseAEstudio al expediente creado antes. Internamente, `ActualizacionEstadoExpedienteService` detecta que el último trámite tiene esa etiqueta y la entidad `Expediente` cambia su estado automáticamente a `ParaResolver`. Se vuelve a listar para verificarlo.

```
Console.WriteLine(
    "== 3. Agregar un tramite y verificar cambio de estado automatico"
    "=="
);

try
{
    var expedientes = listarExpedientes.Ejecutar().ToList();
    var idExpediente = expedientes.First().Id;

    var request = new AgregarTramiteRequest(
        idExpediente,
        EtiquetaTramite.PaseAEstudio,
        "Pase a estudio del expediente",
        idUsuario);

    var response = agregarTramite.Ejecutar(request);

    Console.WriteLine(
        $"[Exito] Tramite agregado. Id: {response.IdTramite} | " +
        $"Etiqueta: {request.Etiqueta}");

    var expedienteActualizado =
        listarExpedientes.Ejecutar()
            .First(e => e.Id == idExpediente);
}
```

```

        Console.WriteLine(
            $"[Verificacion] Estado del expediente: " +
            $"{expedienteActualizado.Estado} " +
            $"(esperado: ParaResolver)\n");
    }
    catch (DominioException ex)
    {
        Console.WriteLine($"[DominioException]: {ex.Message}\n");
    }
    catch (AutorizacionException ex)
    {
        Console.WriteLine($"[AutorizacionException]: {ex.Message}\n");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[Exception]: {ex.Message}\n");
    }
}

```

Código 5: Alta de trámite con cambio de estado automático

```

=== 3. Agregar tramite y verificar cambio de estado automatico ===
[Exito] Tramite agregado. Id: 7c9e6679-7425-40de-944b-e07fc1f90ae7 |
Etiqueta: PaseAEstudio
[Verificacion] Estado: ParaResolver (esperado: ParaResolver)

```

Imagen 4: Escenario Exitoso

```

=== 3. Agregar un tramite y verificar cambio de estado automatico ===
[AutorizacionException]: El usuario no tiene permisos para crear
tramites.

```

Imagen 5: Escenario sin permisos

Prueba 4 — Cambiar estado manualmente

Se cambia el estado del expediente a `EnNotificacion` directamente, sin necesidad de agregar ningún trámite. Esto representa el flujo natural donde el usuario avanza el expediente manualmente.

```

Console.WriteLine("=== 4. Cambiar estado manualmente ===");

try
{
    var idExpediente =
        listarExpedientes.Ejecutar().First().Id;

    var request = new CambiarEstadoRequest(
        idExpediente,
        EstadoExpediente.EnNotificacion,
        idUsuario);

    cambiarEstado.Ejecutar(request);

    Console.WriteLine(
        $"[Exito] Estado cambiado a: {request.NuevoEstado}\n");
}

```

```

catch (DominioException ex)
{
    Console.WriteLine($"[DominioException]: {ex.Message}\n");
}
catch (AutorizacionException ex)
{
    Console.WriteLine($"[AutorizacionException]: {ex.Message}\n");
}
catch (Exception ex)
{
    Console.WriteLine($"[Exception]: {ex.Message}\n");
}

```

Código 6: Cambio de estado manual

```

=== 4. Cambiar estado manualmente ===
[Exito] Estado cambiado a: EnNotificacion

```

Imagen 6: Salida esperada

```

=== 4. Cambiar estado manualmente ===
[AutorizacionException]: El usuario no tiene permisos para modificar el
estado.

```

Imagen 7: Escenario sin permisos

Prueba 5 — Modificar carátula

Se corrige la carátula de un expediente existente. Esta operación no afecta el estado del expediente, solo actualiza el texto y registra quién hizo el cambio y cuándo.

```

Console.WriteLine("=== 5. Modificar caratula ===");

try
{
    var idExpediente =
        listarExpedientes.Ejecutar().First().Id;

    var request = new ModificarCaratulaRequest(
        idExpediente,
        "Caratula corregida",
        idUsuario);

    modificarCaratula.Ejecutar(request);

    Console.WriteLine(
        $"[Exito] Caratula modificada a: {request.NuevaCaratula}\n");
}
catch (DominioException ex)
{
    Console.WriteLine($"[DominioException]: {ex.Message}\n");
}
catch (AutorizacionException ex)
{
    Console.WriteLine($"[AutorizacionException]: {ex.Message}\n");
}

```

Código 7: Modificación de carátula

```
=== 5. Modificar caratula ===  
[Exito] Caratula modificada a: Caratula corregida
```

Imagen 8: Salida esperada

```
=== 5. Modificar caratula ===  
[AutorizacionException]: El usuario no tiene permisos para modificar  
expedientes.
```

Imagen 9: Escenario sin permisos

Caminos de Error

Prueba 6 — Carátula vacía lanza `DominioException`

Si se intenta crear un expediente con carátula vacía, el Value Object `Caratula` lanza una `DominioException` al instanciarse. El `catch` correspondiente la captura e imprime el mensaje.

```
Console.WriteLine("=== 6. Intentar crear un expediente con caratula  
vacía ===");  
try  
{  
    var request = new AgregarExpedienteRequest("", idUsuario);  
    agregarExpediente.Ejecutar(request);  
}  
catch (DominioException ex) { Console.WriteLine($"[DominioException]: {  
    ex.Message}"); }  
catch (AutorizacionException ex) { Console.WriteLine($"[  
    AutorizacionException]: {ex.Message}"); }  
catch (Exception ex) { Console.WriteLine($"[Exception]: {ex.  
    Message}"); }
```

Código 8: Intento de crear expediente con carátula vacía

```
=== 6. Intentar crear un expediente con caratula vacia ===  
[DominioException]: El texto de la caratula es invalido.
```

Imagen 10: Salida esperada

Prueba 7 — ID inexistente lanza `EntidadNoEncontradaException`

Si se intenta operar sobre un expediente con un ID que no existe en el archivo, el repositorio devuelve null y el Caso de Uso lanza una `EntidadNoEncontradaException`.

```
Console.WriteLine("=== 7. Intentar operar con un expediente inexistente  
===");  
try  
{  
    var request = new CambiarEstadoRequest(Guid.NewGuid(),  
        EstadoExpediente.Finalizado, idUsuario);  
    cambiarEstado.Ejecutar(request);  
}  
catch (EntidadNoEncontradaException ex) { Console.WriteLine($"[  
    EntidadNoEncontradaException]: {ex.Message}"); }
```

```
catch (AutorizacionException ex) { Console.WriteLine($"[
    AutorizacionException]: {ex.Message}"); }
```

Código 9: Operación sobre expediente inexistente

```
=== 7. Intentar operar con un expediente inexistente ===
[EntidadNoEncontradaException]: No se encontro el expediente con Id 9
b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d.
```

Imagen 11: Salida esperada

Prueba 8 — AutorizacionException al denegar permisos

Para probar esta excepción hay que ir a `AutorizacionProvisionalService.cs` y cambiar `return true` por `return false`. Al volver a ejecutar, todos los Casos de Uso que realizan modificaciones lanzarán `AutorizacionException` antes de realizar cualquier operación.

```
// SGE.Infraestructura/AutorizacionProvisionalService.cs
public bool PoseeElPermiso(Guid idUsuario, Permiso permiso)
{
    return false; // Cambiar a false para probar AutorizacionException
}
```

Código 10: Cambio en `AutorizacionProvisionalService` para probar la excepción

```
=== 1. Agregar un expediente valido ===
[AutorizacionException]: El usuario no tiene permiso para crear
expedientes.
```

Imagen 12: Salida esperada al denegar permisos

Recordar volver a `return true` después de probar esta funcionalidad para que el resto de las pruebas funcionen correctamente.